

Міністерство освіти і науки України  
Чернівецький національний університет імені Юрія Федьковича  
Кафедра комп'ютерних наук

**ТЕХНІЧНИЙ ДОКУМЕНТ**  
**інтелектуальної інформаційної системи**  
**«Self-Healing Pipeline»**

самовідновлювальний конвеєр аналізу тональності текстових відгуків

Предметна область: DataOps / обробка природної мови (NLP)

**Лабораторна робота № 10**  
**«Формування структури технічного документа для власного проєкту»**  
з дисципліни «Проектування інтелектуальних інформаційних систем»

Виконав: студент 4-го курсу, групи 444а  
спеціальності 122 Комп'ютерні науки  
**Мулик А.В.**

Перевірив:  
**к.ф.-м.н., доцент, Ковальчук М.Л.**

Чернівці –2026 н.р.

## **Анотація**

Self-Healing Pipeline - інтелектуальна інформаційна система для автоматизованої обробки та аналізу тональності великих обсягів текстових відгуків (на прикладі Yelp Academic Dataset, понад 6 ГБ). Система призначена для аналітиків даних (Data/Business Analyst) та DevOps/Data-інженерів, які потребують надійного, відмовостійкого конвеєра обробки краудсорсингових текстових даних без передачі конфіденційної інформації хмарним провайдером.

Основна ML-задача системи - класифікація тексту (Sentiment Analysis, задача обробки природної мови) на три класи тональності (POSITIVE / NEGATIVE / NEUTRAL), яка вирішується через zero-shot інференс локальної великої мовної моделі llama3.2 (Ollama REST API) без донавчання на доменних даних. Архітектурно система поєднує оркестрований конвеєр Apache Airflow з детерміністичним модулем самовідновлення (виявлення та виправлення 5 типів дефектів вхідних даних), колонкове аналітичне сховище DuckDB та вебдашборд на основі Blazor WebAssembly для моніторингу й аудиту запусків.

## **Специфікація вимог (SRS, скорочений варіант)**

### **Вступ**

Назва системи: Self-Healing Pipeline.

Мета документа: надати повний технічний опис архітектури, вимог та архітектурно значущих рішень інтелектуальної інформаційної системи, розробленої в межах курсу «Проектування інтелектуальних інформаційних систем». Документ узагальнює результати практичних робіт 1–8 курсу та може слугувати основою для розділів пояснювальної записки кваліфікаційної роботи бакалавра «Розробка самовідновлювального конвеєру ETL та аналітичного сховища для аналізу текстових відгуків з використанням Apache Airflow і локальних великих мовних моделей».

### **Загальний опис системи**

Актори (користувачі) системи:

- Data/Business Analyst - обирає конкретний бізнес із датасету, запускає аналіз його відгуків, переглядає health-звіти, аналітичні графіки та кореляцію зірок із тональністю.

- DevOps / Data Engineer - налаштовує параметри DAG-ів Apache Airflow (batch\_size, ollama\_model), розгортає й обслуговує інфраструктуру (Docker, Ollama).
- Адміністратор системи - керує обліковими записами користувачів вебдашборду через механізм автентифікації.

Проблема, яку вирішує система: реальні краудсорсингові датасети текстових відгуків систематично містять дефекти якості (відсутні значення, хибні типи, порожні чи надмірно довгі тексти), що унеможлиблює їх пряму класифікацію. Водночас хмарні сервіси аналізу тональності вимагають передачі тексту відгуків зовнішнім провайдерам, що порушує вимоги конфіденційності. Система розв'язує обидві проблеми одночасно: автоматично виправляє дефекти вхідних даних і класифікує тональність локально, без використання хмарних LLM-провайдерів.

Операційне середовище: серверна частина (Apache Airflow) розгортається у Docker-контейнері (режим standalone) на хост-машині Windows/Linux; сервіс Ollama виконується як процес на хост-машині (доступний контейнеру через host.docker.internal:11434); вебдашборд (.NET 8, ASP.NET Core + Blazor WebAssembly) запускається командою dotnet run; клієнтська частина (Blazor WASM) виконується безпосередньо в браузері користувача.

## Функціональні вимоги

Кожна вимога сформульована за шаблоном «Система повинна [дія] [об'єкт] [умова або обмеження]», має унікальний ідентифікатор та піддається перевірці тест-кейсом (див. розділ «Матриця трасованості вимог»).

| ID     | Формулювання вимоги                                                                                                                                                                                                                           |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-001 | Система повинна автоматично виявляти та виправляти щонайменше 5 типів дефектів вхідного тексту (відсутній текст, хибний тип даних, порожній текст, текст лише зі спецсимволів, текст довжиною понад 2000 символів) без зупинки обробки батчу. |
| FR-002 | Система повинна виконувати пакетну обробку відгуків із конфігурованими параметрами запуску (batch_size, offset, ollama_model) через параметризовані DAG Apache Airflow (Param API).                                                           |
| FR-003 | Система повинна класифікувати тональність кожного відгуку через REST-запит до Ollama (POST /api/chat), повертаючи результат у форматі JSON із полями sentiment (POSITIVE/NEGATIVE/NEUTRAL) та confidence (0,0–1,0).                           |

|        |                                                                                                                                                                                                                                 |
|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FR-004 | Система повинна виконувати до 3 повторних спроб запиту до Ollama при таймауті 120 секунд та застосовувати резервний (fallback) парсинг ключових слів за неможливості розпарсити JSON-відповідь.                                 |
| FR-005 | Система повинна переводити запис у деградований стан (status=degraded, sentiment=NEUTRAL, confidence=0,5) при недоступності сервісу Ollama після вичерпання спроб, без переривання виконання DAG.                               |
| FR-006 | Система повинна формувати після кожного запуску health-звіт у форматі JSON з інтегральним статусом (одне зі значень HEALTHY/WARNING/DEGRADED/CRITICAL), обчисленим на основі healing_rate та degradation_rate.                  |
| FR-007 | Система повинна надавати REST API та вебінтерфейс (Blazor) для перегляду списку запусків, детальної аналітики обраного запуску (відгуки, статистика, кореляція зірок/тональності) та експорту відгуків бізнесу у формат NDJSON. |
| FR-008 | Система повинна автентифікувати користувача через ендпоінт POST /api/auth/login, видаючи Bearer-токен, дійсність якого перевіряється для кожного запиту до захищених ендпоінтів.                                                |

## Нефункціональні вимоги та вимоги до якості ML-компонента

| ID      | Категорія                  | Формулювання вимоги                                                                                                                                                                                                                                                                                                                                  |
|---------|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-001 | Якість ML-компонента       | Середній рівень впевненості (confidence) класифікації для записів зі статусом success повинен бути не нижче 0,75; частка узгодження передбаченої тональності з екстремальними значеннями зіркового рейтингу (1 зірка→NEGATIVE, 5 зірок →POSITIVE, як проксі-міра якості за відсутності розміченого тестового набору) повинна становити не менше 75%. |
| NFR-002 | Продуктивність (затримка)  | Середній час інференсу тональності одного відгуку через Ollama (llama3.2, CPU) не повинен перевищувати 6 секунд при типовому розмірі тексту відгуку (≤500 символів).                                                                                                                                                                                 |
| NFR-003 | Продуктивність (аналітика) | Виконання аналітичного SQL-запиту до DuckDB по датасету обсягом понад 6 ГБ повинно завершуватися за час не більше 500 мс.                                                                                                                                                                                                                            |

|         |                             |                                                                                                                                                                                                                  |
|---------|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| NFR-004 | Надійність /<br>доступність | Відмова зовнішнього сервісу Ollama не повинна призводити до падіння DAG: конвеєр повинен завершувати виконання успішно (graceful degradation) у 100% випадків недоступності сервісу, фіксуючи це у health-звіті. |
| NFR-005 | Безпека                     | Доступ до всіх захищених REST-ендпоінтів повинен вимагати дійсний Bearer-токен; паролі користувачів повинні зберігатися виключно у хешованому вигляді, без збереження у відкритому тексті.                       |

## Зовнішні інтерфейси

- Ollama REST API - HTTP POST `http://localhost:11434/api/chat` (або `host.docker.internal:11434` з контейнера); формат запиту/відповіді - JSON (`{model, messages}` → `{sentiment, confidence}`).
- Airflow REST API v2 - HTTP; ендпоінти запуску DAG (`trigger-dag`), отримання статусу запуску (`dag-run/{runId}`), станів задач (`task-instances`) та логів (`task-logs`); формат - JSON.
- Yelp Academic Dataset - файлове джерело даних (JSON, понад 6 ГБ), що одноразово імпортується через `YelpImportService` у DuckDB.
- DuckDB.NET - вбудована (in-process) бібліотека доступу до аналітичного сховища DuckDB; SQL-інтерфейс, не мережевий протокол.
- Browser ↔ Dashboard Backend - HTTPS, REST API (JSON) для даних та статичні файли WebAssembly для завантаження клієнтського застосунку Blazor.

## Архітектурні діаграми

У цьому розділі наведено п'ять архітектурних діаграм системи Self-Healing Pipeline, побудованих у нотації PlantUML/C4/UML. Кожна діаграма супроводжується підписом та текстовим поясненням ключових архітектурних рішень, відображених на ній, із посиланням на відповідний запис ADR (розділ 5).

Рисунок 1 - Context Diagram (C4 Level 1)

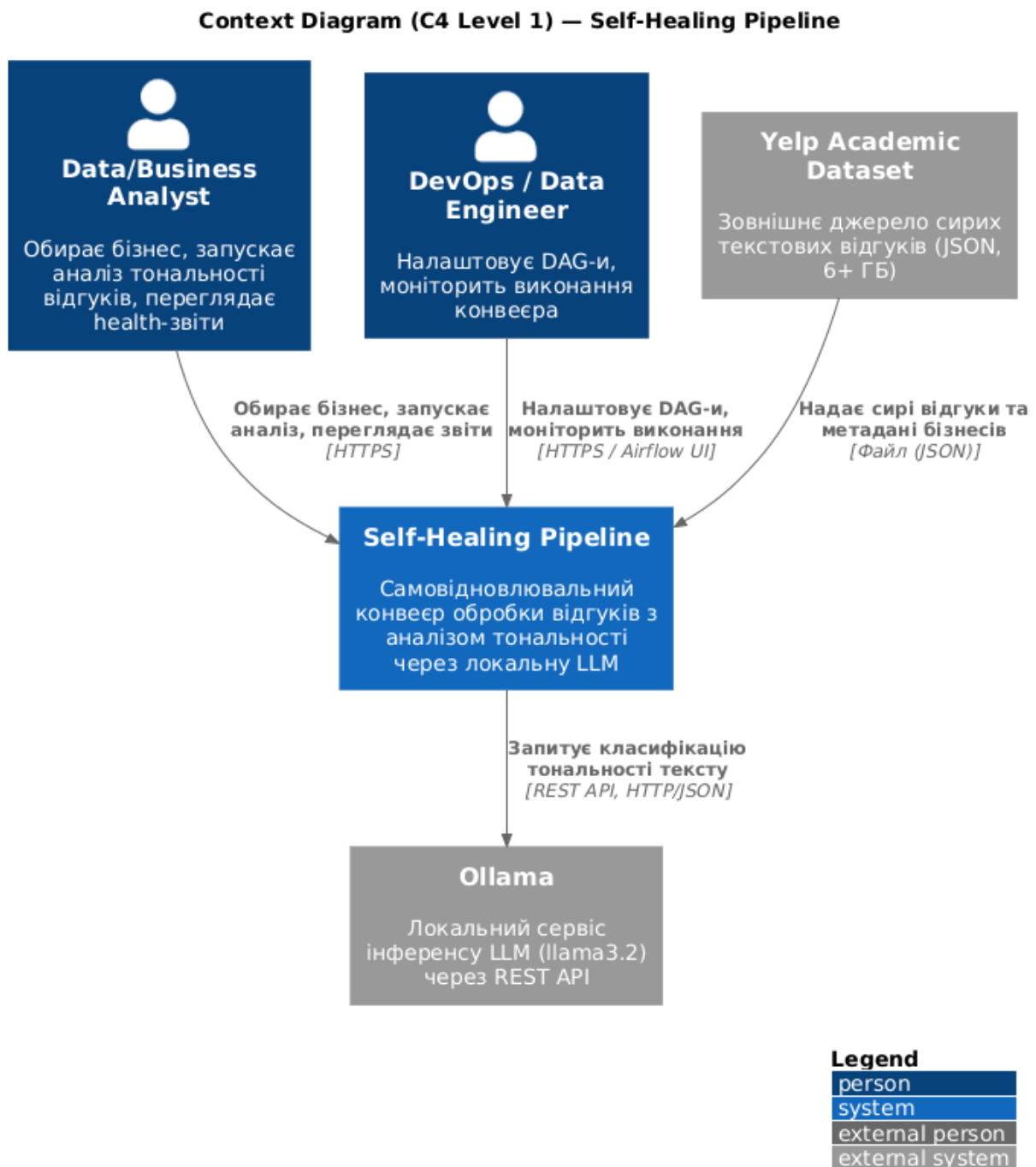


Рисунок 1 - Context Diagram системи Self-Healing Pipeline (C4 рівень 1)

Діаграма відображає систему на найвищому рівні абстракції: два актори (Data/Business Analyst, DevOps/Data Engineer) та дві зовнішні системи (Ollama, Yelp Academic Dataset), що лежать поза межею відповідальності Self-Healing Pipeline. Уся внутрішня логіка самовідновлення, аналізу тональності та зберігання даних прихована всередині єдиного блоку системи - це безпосередньо відображає архітектурне рішення ADR-001 щодо двокомпонентного модульного моноліту: зовні система презентується як єдиний цілісний блок, незалежно від внутрішнього поділу на Python/Airflow- та .NET/Blazor-підсистеми.

## Рисунок 2 - Component Diagram

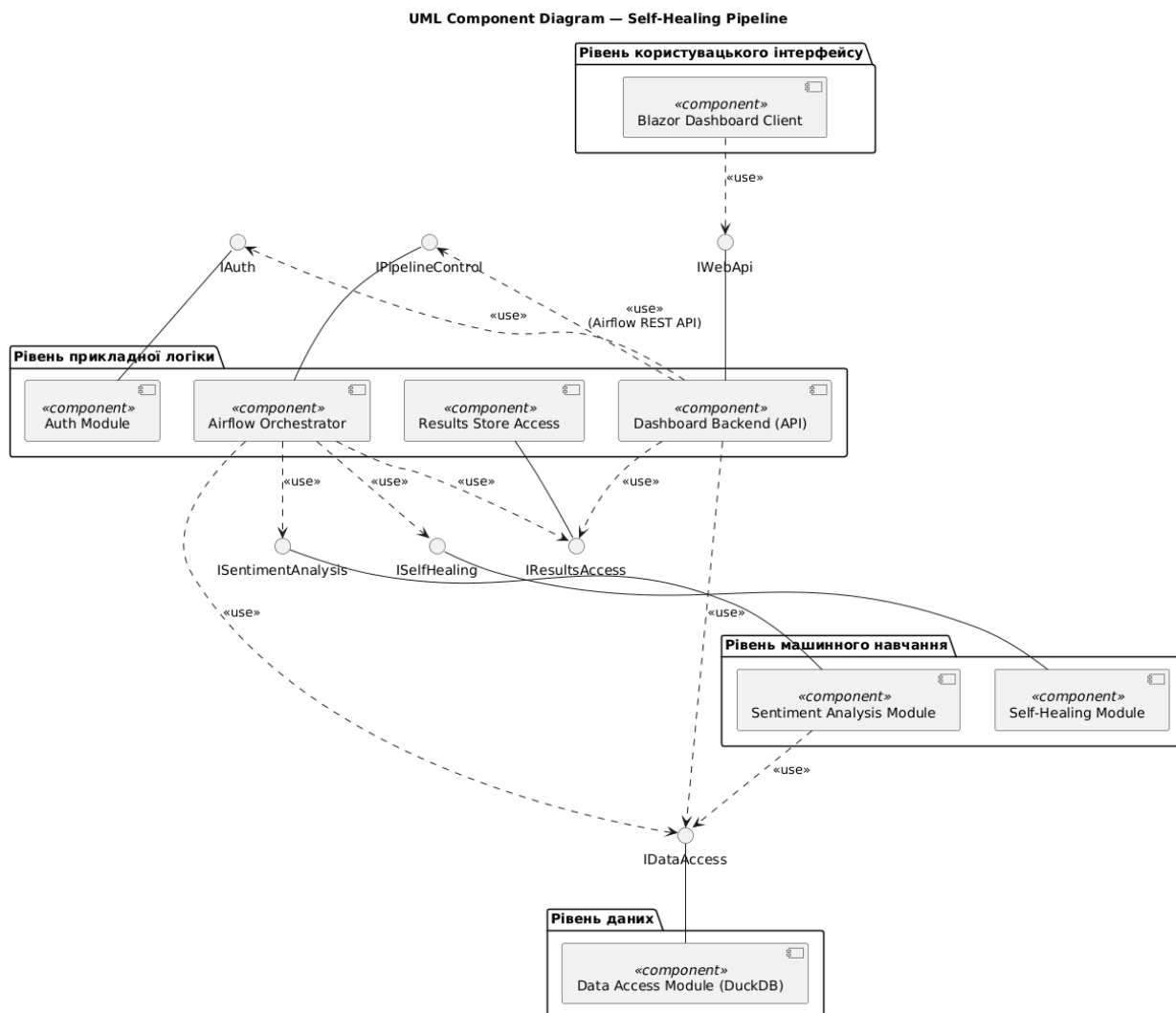


Рисунок 2 - UML Component Diagram системи Self-Healing Pipeline

Діаграма деталізує внутрішню структуру системи до дев'яти компонентів, згрупованих за чотирма рівнями (UI, прикладна логіка, машинне навчання, дані), з явним відображенням provided/required інтерфейсів. Виокремлення Self-Healing Module та Sentiment Analysis Module в окремі компоненти рівня машинного навчання відповідає рішення ADR-004 (детерміністичний rule-based self-healing) - обидва модулі незалежні один від одного й взаємодіють із рештою системи лише через інтерфейси ISelfHealing/ISentimentAnalysis. Наявність окремого Auth Module з інтерфейсом IAuth відображає рішення ADR-005 щодо власної реалізації автентифікації на основі Bearer-токенів, а централізований Data Access Module (DuckDB) - рішення ADR-002 щодо єдиної точки доступу до аналітичного сховища.

Рисунок 3 - ML Pipeline (Self-Healing + Sentiment Analysis)

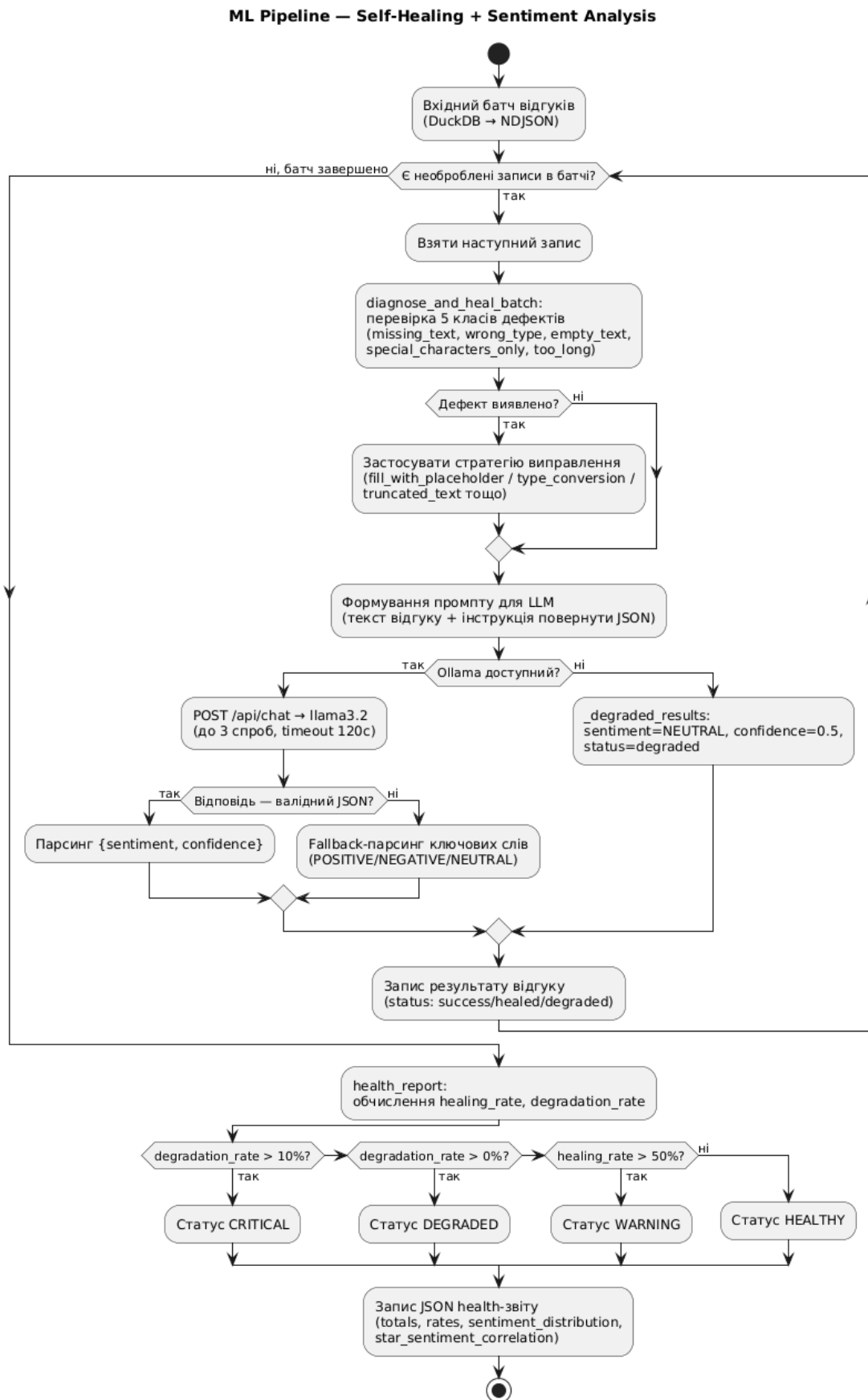


Рисунок 3 - ML Pipeline: самовідновлення та аналіз тональності

Діаграма активності відображає повний алгоритм обробки одного батчу: для кожного запису спочатку виконується перевірка п'яти класів дефектів і застосування відповідної стратегії виправлення (функція `_heal_review`, патерн «Ланцюжок відповідальності» - ADR-004), після чого формується промпт і виконується звернення до Оллани. Цикл розгалуження «Оллана доступний? / Відповідь валідна?» з гілками `fallback-парсингу` та `_degraded_results` безпосередньо реалізує рішення ADR-003 щодо пакетного синхронного інференсу з обробкою відмов на рівні кожного запиту, а не на рівні всього батчу. Завершальний блок `health_report` обчислює інтегральний статус конвєсера (HEALTHY/WARNING/DEGRADED/CRITICAL) на основі накопичених часток `healing_rate` і `degradation_rate` (вимога FR-006).

Рисунок 4 - Deployment Diagram

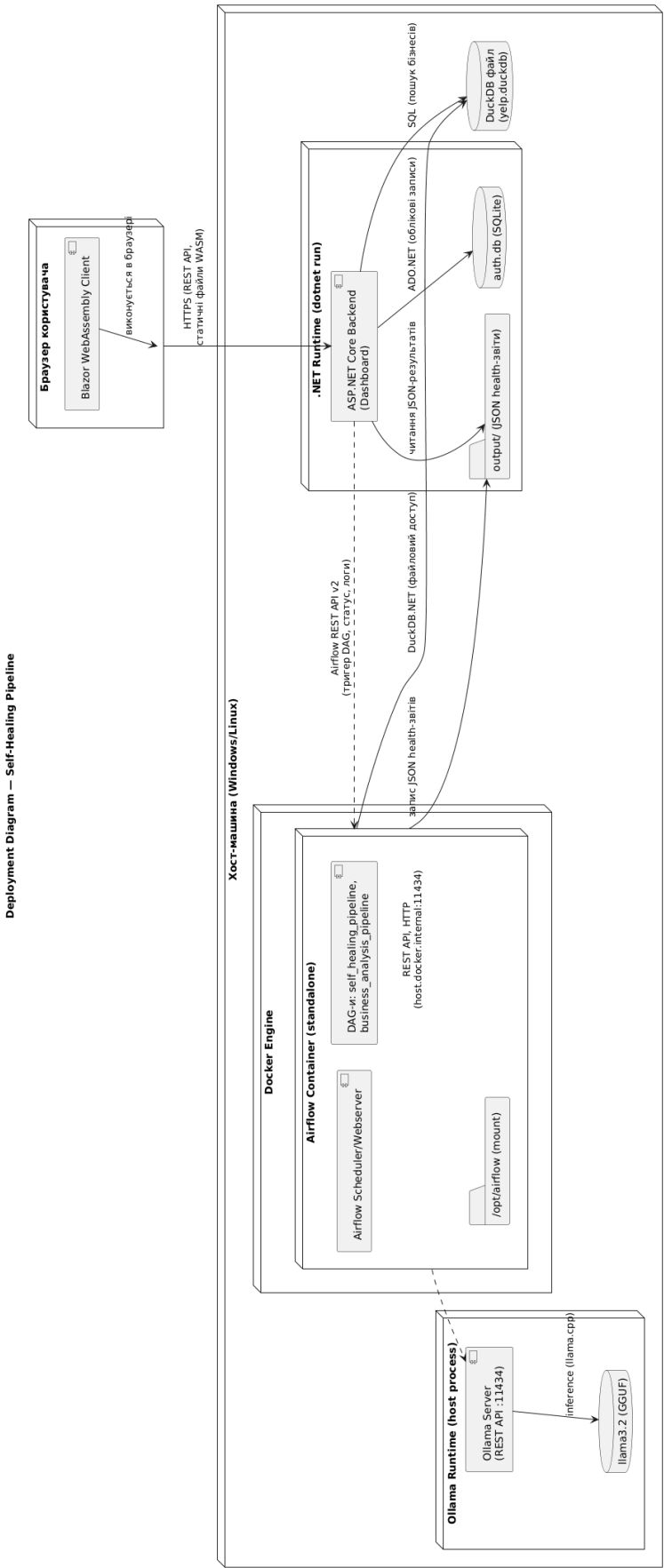


Рисунок 4 - Deployment Diagram системи Self-Healing Pipeline

Діаграма розгортання показує фізичний розподіл компонентів по трьох виконавчих середовищах на хост-машині: Docker-контейнер з Airflow (standalone, з примонтованою директорією /opt/airflow), окремий хост-процес Ollama (доступний контейнеру через host.docker.internal:11434) та .NET-процес вебдашборду (dotnet run) із власними файлами auth.db (SQLite) та output/ (JSON-результати). Таке розділення на три незалежно розгортвані середовища є прямим наслідком рішень ADR-001 (двокомпонентна архітектура без мікросервісів) та ADR-002 (DuckDB як вбудована, а не серверна СУБД - файл yelp.duckdb доступний обом підсистемам напряму, без окремого процесу бази даних). Клієнт Blazor WebAssembly довантажується в браузер користувача та виконується повністю на стороні клієнта.

### Рисунок 5 - Sequence Diagram

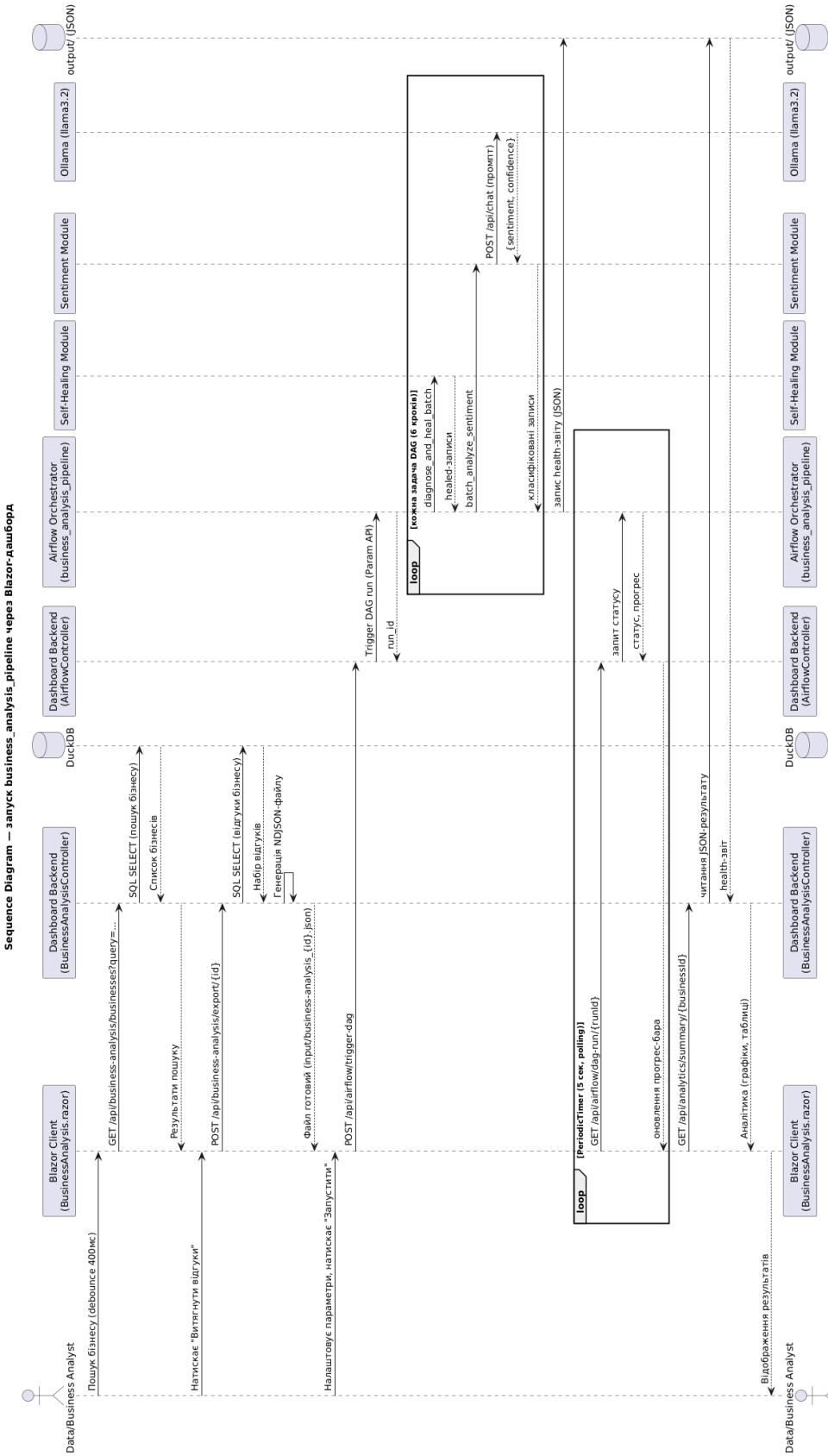


Рисунок 5 - Sequence Diagram: запуск business\_analysis\_pipeline через Blazor-дашборд

Діаграма послідовності демонструє наскрізний сценарій: пошук бізнесу та експорт його відгуків через SQL-запити до DuckDB (ADR-002), запуск DAG через Airflow REST

API, послідовну обробку записів усередині DAG (цикл `diagnose_and_heal_batch` → `batch_analyze_sentiment` → Ollama, що ілюструє пакетний синхронний інференс ADR-003) та відстеження прогресу через періодичний polling (PeriodicTimer, 5 секунд) із боку клієнта. Завершальний блок отримання аналітики (GET `/api/analytics/summary`) читає вже записаний health-звіт (output/) і повертає агреговані дані для відображення користувачу, що відповідає вимозі FR-007.

## **Колекція ADR (Architecture Decision Records)**

Нижче наведено п'ять записів архітектурних рішень (ADR) за класичним шаблоном Nygard, що охоплюють справжні альтернативи, розглянуті під час проєктування системи Self-Healing Pipeline протягом курсу.

### **ADR-001. Вибір архітектурного стилю - двокомпонентний модульний моноліт замість мікросервісів**

Status: Accepted

Context. Система складається з принципово різнотехнологічних підсистем - конвеєра обробки даних (Python/Apache Airflow) та вебдашборду (C#/.NET Blazor). Виникло питання: виокремлювати кожен функціональний блок (самовідновлення, аналіз тональності, автентифікація, аналітика) в окремий мікросервіс, чи об'єднати їх у небагато великих компонентів у межах одного репозиторію.

Considered Options:

1. Повноцінна мікросервісна архітектура (окремий сервіс на кожен логічний компонент, комунікація через черги/REST).
2. Модульний моноліт у межах двох підсистем (Airflow-конвеєр + .NET-дашбورد), що взаємодіють через спільну файлову систему/БД та REST.
3. Єдиний моноліт на одній технологічній платформі (відмова від Python або від .NET).

Decision. Обрано варіант 2 - двокомпонентна архітектура в одному монорепозіторії: Python/Airflow-підсистема для обробки даних та .NET/Blazor-підсистема для вебмоніторингу, що обмінюються даними через DuckDB, JSON-файли результатів та Airflow REST API.

Consequences. Позитивні: низька операційна складність порівняно з мікросервісами (немає потреби в оркестраторі контейнерів, service mesh, розподілених транзакціях); кожна

підсистема використовує технологію, найбільш природну для своєї задачі. Прийняті компроміси: підсистеми не можуть незалежно масштабуватися горизонтально; еволюційний поділ на справжні мікросервіси в майбутньому потребуватиме рефакторингу меж компонентів.

## **ADR-002. Вибір аналітичного сховища даних - DuckDB замість прямого читання JSON або серверної реляційної СУБД**

Status: Accepted

Context. Вхідний датасет Yelp Academic Dataset має обсяг понад 6 ГБ у форматі JSON. Пряме послідовне читання файлу для кожного аналітичного запиту (наприклад, пошук бізнесу) є неприйнятно повільним (хвилини на запит), а традиційна серверна СУБД додає операційну складність для навчального/демонстраційного стенду.

Considered Options:

1. Пряме читання та парсинг JSON-файлів при кожному запиті.
2. Серверна row-based реляційна СУБД (PostgreSQL/MySQL) з імпортом даних у таблиці.
3. Колонкова вбудована (embedded) OLAP-СУБД DuckDB.

Decision. Обрано DuckDB як основне аналітичне сховище для Yelp-датасету завдяки колонковому формату зберігання, векторизованому виконанню запитів та zone тар-фільтрації. SQLite залишено окремо - виключно для службових облікових даних автентифікації (auth.db), де профіль навантаження принципово інший (точкові вставки/вибірki, а не аналітичні агрегації).

Consequences. Позитивні: субсекундний (мілісекунди) час виконання аналітичних запитів навіть на датасеті 10+ ГБ; відсутність потреби в окремому серверному процесі СУБД. Прийняті компроміси: DuckDB менш придатна для високочастотних точкових транзакційних записів (тому auth-дані свідомо винесено в SQLite); додаткова залежність (DuckDB.NET binding) у .NET-частині системи.

## **ADR-003. Режим інференсу ML-компонента - пакетний синхронний виклик Ollama без онлайн-стрімінгу**

Status: Accepted

Context. Класифікація тональності кожного відгуку вимагає звернення до LLM (приблизно 5 секунд на запит на CPU). Дані надходять не в реальному часі, а пакетами (з

датасету або з експортованого NDJSON конкретного бізнесу), тому виникло питання щодо режиму інференсу.

Considered Options:

1. Онлайн (real-time) інференс - класифікація одразу при надходженні кожного окремого відгуку через стрімінговий API.
2. Пакетний (batch) послідовний інференс у межах однієї задачі Airflow DAG.
3. Пакетний інференс із паралельними запитами до Ollama (черга/пул воркерів) усередині одного батчу.

Decision. Обрано пакетний послідовний інференс (варіант 2) у межах задачі `batch_analyze_sentiment/analyze_sentiment` DAG; паралелізм винесено на рівень кількох одночасних DAG-запусків через `batch_runner.py` (`ThreadPoolExecutor`), а не на рівень окремих запитів до Ollama в межах одного батчу.

Consequences. Позитивні: простота реалізації та передбачуваність навантаження на Ollama (один активний запит у межах задачі); природна узгодженість із batch-орієнтованою моделлю Airflow DAG. Прийняті компроміси: крок `analyze_sentiment` домінує за часом виконання (96–98% загального часу DAG); система не придатна для сценаріїв, що вимагають миттєвої (sub-second) класифікації окремого відгуку в момент надходження.

#### **ADR-004. Стратегія самовідновлення даних - детерміністичні правила (rule-based) замість навчального (RL-based) підходу**

Status: Accepted

Context. Вхідні дані Yelp-датасету систематично містять п'ять типів дефектів. Потрібно було обрати механізм автоматичного виправлення: статичні правила чи модель, що навчається оптимальній стратегії виправлення (наприклад, на основі підкріплення).

Considered Options:

1. ML-based самовідновлення на основі підкріплення (RL, напр. DQN/PPO) - агент навчається оптимальній стратегії виправлення.
2. Meta-learning (MAML) підхід для швидкої адаптації до нових типів дефектів за малою кількістю прикладів.
3. Детерміністичний rule-based підхід - явне відображення «тип дефекту → стратегія виправлення».

Decision. Обрано детерміністичний rule-based підхід (варіант 3), реалізований через послідовну перевірку п'яти умов у функції `_heal_review` за патерном «Ланцюжок відповідальності».

Consequences. Позитивні: повна передбачуваність і відтворюваність результату для будь-якої комбінації вхідних даних (критично для аудитабельності); не вимагає навчальних даних чи обчислювальних ресурсів для тренування агента; проста верифікація через unit-тести для кожного правила. Прийняті компроміси: система не адаптується автоматично до нових, наперед не передбачених типів дефектів - додавання нового правила вимагає ручної зміни коду.

## **ADR-005. Механізм автентифікації - власні Bearer-токени та SQLite замість зовнішнього OAuth/SSO-провайдера**

Status: Accepted

Context. Вебдашборд потребує розмежування доступу до захищених ендпоінтів (запуск DAG, перегляд даних) для обмеженої кількості внутрішніх користувачів навчального/демонстраційного стенду.

Considered Options:

1. Інтеграція із зовнішнім OAuth2/OpenID Connect провайдером (напр. Auth0, Keycloak, Microsoft Entra ID).
2. Власна реалізація автентифікації на основі Bearer-токенів зі зберіганням облікових записів у SQLite (AuthController + TokenAuthHandler).
3. Базова HTTP Authentication (логін/пароль у кожному запиті без токена).

Decision. Обрано власну реалізацію Bearer-токенів (варіант 2): AuthController видає токен при успішному вході, TokenAuthHandler валідує його на кожному запиті, AuthStateService зберігає токен на клієнті (Blazor) у пам'яті сесії.

Consequences. Позитивні: відсутність зовнішньої залежності та витрат на сторонній SSO-сервіс, що є доречним для проєкту з обмеженою кількістю користувачів; повний контроль над логікою токенів. Прийняті компроміси: відсутні готові механізми зовнішнього провайдера (MFA, соціальний логін, централізоване управління користувачами); масштабування на велику кількість зовнішніх користувачів потребуватиме переходу на стандартний OAuth2/OIDC.

## Матриця трасованості вимог (RTM)

Таблиця пов'язує всі функціональні та нефункціональні вимоги (розділи 2.3, 2.4) з компонентами архітектури (розділ 3) та відповідними тест-кейсами.

| ID вимоги | Компонент архітектури                | Тест-кейс                                                |
|-----------|--------------------------------------|----------------------------------------------------------|
| FR-001    | Self-Healing Module                  | ТС-001 - перевірка виправлення 5 типів дефектів          |
| FR-002    | Airflow Orchestrator                 | ТС-002 - запуск DAG з різними batch_size/offset          |
| FR-003    | Sentiment Analysis Module            | ТС-003 - коректний парсинг JSON-відповіді Ollama         |
| FR-004    | Sentiment Analysis Module            | ТС-004 - некоректна відповідь → fallback-парсинг         |
| FR-005    | Sentiment Analysis Module            | ТС-005 - вимкнена Ollama → статус degraded               |
| FR-006    | Airflow Orchestrator (health_report) | ТС-006 - коректність обчислення health-статусу           |
| FR-007    | Dashboard Backend / Blazor Client    | ТС-007 - перегляд списку запусків і деталей              |
| FR-008    | Auth Module                          | ТС-008 - видача та валідація Bearer-токену               |
| NFR-001   | Sentiment Analysis Module            | ТС-009 - вимірювання confidence і кореляції з зірками    |
| NFR-002   | Sentiment Analysis Module / Ollama   | ТС-010 - вимірювання часу інференсу одного відгуку       |
| NFR-003   | Data Access Module (DuckDB)          | ТС-011 - бенчмарк SQL-запиту на повному датасеті         |
| NFR-004   | Airflow Orchestrator                 | ТС-012 - відмовостійкість при недоступному Ollama        |
| NFR-005   | Auth Module                          | ТС-013 - доступ без токена / перевірка хешування паролів |

## Глосарій

| Термін                         | Визначення                                                                                                                                        |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Bearer-токен                   | Рядок автентифікації, що передається в заголовок Authorization HTTP-запиту для підтвердження прав доступу до захищених ендпоінтів.                |
| DAG (Directed Acyclic Graph)   | Спрямований ациклічний граф задач - базова одиниця оркестрації робочого процесу в Apache Airflow.                                                 |
| DuckDB                         | Вбудована (embedded) колонкова OLAP-СУБД, що використовується як основне аналітичне сховище Yelp-датасету.                                        |
| ETL                            | Extract, Transform, Load - процес витягнення, трансформації та завантаження даних.                                                                |
| Graceful degradation           | Принцип, за яким система продовжує працювати в обмеженому («деградованому») режимі при відмові зовнішнього сервісу, а не завершується з помилкою. |
| Health-звіт                    | Структурований JSON-документ з агрегованою статистикою виконання та інтегральним статусом запуску конвеєра (HEALTHY/WARNING/DEGRADED/CRITICAL).   |
| LLM (Large Language Model)     | Велика мовна модель; у системі використовується локальна модель llama3.2 через Ollama.                                                            |
| NDJSON                         | Newline Delimited JSON - формат, у якому кожен рядок файлу є окремим валідним JSON-об'єктом; використовується для експорту відгуків бізнесу.      |
| Ollama                         | Відкрита інфраструктура для локального запуску великих мовних моделей через CLI та REST API (порт 11434 за замовчуванням).                        |
| OLAP                           | Online Analytical Processing - клас систем, оптимізованих для аналітичних (а не транзакційних) запитів до великих обсягів даних.                  |
| Self-healing (самовідновлення) | Здатність конвеєра автоматично виявляти й виправляти дефекти вхідних даних без зупинки обробки та без втрати контексту виконання.                 |
| Sentiment Analysis (SA)        | Аналіз тональності тексту - задача NLP, що полягає у визначенні емоційного забарвлення тексту (POSITIVE/NEGATIVE/NEUTRAL).                        |
| TaskFlow API                   | Декларативний спосіб визначення DAG та задач Apache Airflow через декоратори @dag і @task з явною передачею даних між задачами.                   |
| WASM (WebAssembly)             | Портативний формат бінарного коду, що дозволяє виконувати .NET-рантайм (Blazor) безпосередньо в браузері без сервера.                             |
| Zero-shot класифікація         | Класифікація моделлю без додаткового донавчання на розмічених прикладах конкретного домену; ключова властивість LLM, застосована в системі.       |

## Додаток. Коди архітектурних діаграм (PlantUML)

Нижче наведено повний вихідний код усіх п'яти діаграм розділу 3 у нотації PlantUML (з розширенням C4-PlantUML для Context Diagram), що дозволяє відтворити або редагувати кожну діаграму.

### Додаток 1. Context Diagram (C4 Level 1)

```
@startuml ContextDiagram SelfHealingPipeline
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Context.puml
LAYOUT_WITH_LEGEND()
title Context Diagram (C4 Level 1) - Self-Healing Pipeline

Person(analyst, "Data/Business Analyst", "Обирає бізнес, запускає аналіз тональності відгуків, переглядає health-звіти")
Person(devOps, "DevOps / Data Engineer", "Налаштовує DAG-и, моніторить виконання конвеєра")

System(selfHealingPipeline, "Self-Healing Pipeline", "Самовідновлювальний конвеєр обробки відгуків з аналізом тональності через локальну LLM")

System_Ext(ollama, "Ollama", "Локальний сервіс інференсу LLM (llama3.2) через REST API")
System_Ext(yelpDataset, "Yelp Academic Dataset", "Зовнішнє джерело сирих текстових відгуків (JSON, 6+ ГБ)")

Rel(analyst, selfHealingPipeline, "Обирає бізнес, запускає аналіз, переглядає звіти", "HTTPS")
Rel(devOps, selfHealingPipeline, "Налаштовує DAG-и, моніторить виконання", "HTTPS / Airflow UI")
Rel(yelpDataset, selfHealingPipeline, "Надає сирі відгуки та метадані бізнесів", "Файл (JSON)")
Rel(selfHealingPipeline, ollama, "Запитує класифікацію тональності тексту", "REST API, HTTP/JSON")
@enduml
```

### Додаток 2. Component Diagram

```
@startuml ComponentDiagram SelfHealingPipeline
title UML Component Diagram - Self-Healing Pipeline

package "Рівень користувацького інтерфейсу" {
    component "Blazor Dashboard Client" <<component>> as BlazorClient
}

package "Рівень прикладної логіки" {
    component "Dashboard Backend (API)" <<component>> as DashboardApi
    component "Airflow Orchestrator" <<component>> as Orchestrator
    component "Auth Module" <<component>> as AuthModule
    component "Results Store Access" <<component>> as ResultsAccess
}

package "Рівень машинного навчання" {
    component "Self-Healing Module" <<component>> as SelfHealing
    component "Sentiment Analysis Module" <<component>> as SentimentModule
}

package "Рівень даних" {
    component "Data Access Module (DuckDB)" <<component>> as DataAccess
}
```

```

}

interface "IWebApi" as IWebApi
interface "IPipelineControl" as IPipelineControl
interface "IAuth" as IAuth
interface "IResultsAccess" as IResultsAccess
interface "ISelfHealing" as ISelfHealing
interface "ISentimentAnalysis" as ISentimentAnalysis
interface "IDataAccess" as IDataAccess

DashboardApi -up- IWebApi
Orchestrator -up- IPipelineControl
AuthModule -up- IAuth
ResultsAccess -up- IResultsAccess
SelfHealing -up- ISelfHealing
SentimentModule -up- ISentimentAnalysis
DataAccess -up- IDataAccess

BlazorClient ..> IWebApi : <<use>>

DashboardApi ..> IAuth : <<use>>
DashboardApi ..> IPipelineControl : <<use>> (Airflow REST API)
DashboardApi ..> IResultsAccess : <<use>>
DashboardApi ..> IDataAccess : <<use>>

Orchestrator ..> ISelfHealing : <<use>>
Orchestrator ..> ISentimentAnalysis : <<use>>
Orchestrator ..> IDataAccess : <<use>>
Orchestrator ..> IResultsAccess : <<use>>

SentimentModule ..> IDataAccess : <<use>>
@enduml

```

### Додаток 3. ML Pipeline

```

@startuml MLPipeline_SelfHealingPipeline
title ML Pipeline - Self-Healing + Sentiment Analysis

start
:Вхідний батч відгуків (DuckDB → NDJSON);

while (Є необроблені записи в батчі?) is (так)
  :Взяти наступний запис;
  :diagnose_and_heal_batch: перевірка 5 класів дефектів
  (missing_text, wrong_type, empty_text,
  special_characters_only, too_long);

  if (Дефект виявлено?) then (так)
    :Застосувати стратегію виправлення
    (fill_with_placeholder / type_conversion / truncated_text тощо);
  else (ні)
  endif

  :Формування промпту для LLM
  (текст відгуку + інструкція повернути JSON);

  if (Ollama доступний?) then (так)
    :POST /api/chat → llama3.2 (до 3 спроб, timeout 120с);
    if (Відповідь - валідний JSON?) then (так)
      :Парсинг {sentiment, confidence};
    else (ні)
      :Fallback-парсинг ключових слів (POSITIVE/NEGATIVE/NEUTRAL);
    endif
  else (ні)
    :_degraded_results: sentiment=NEUTRAL, confidence=0.5, status=degraded;
  end
end

```

```

endif

:Запис результату відгуку (status: success/healed/degraded);
endwhile (ні, батч завершено)

:health_report: обчислення healing_rate, degradation_rate;

if (degradation_rate > 10%?) then (так)
    :Статус CRITICAL;
elseif (degradation_rate > 0%?) then (так)
    :Статус DEGRADED;
elseif (healing_rate > 50%?) then (так)
    :Статус WARNING;
else (ні)
    :Статус HEALTHY;
endif

:Запис JSON health-звіту
(totals, rates, sentiment_distribution, star_sentiment_correlation);
stop
@enduml

```

## Додаток 4. Deployment Diagram

```

@startuml DeploymentDiagram SelfHealingPipeline
title Deployment Diagram - Self-Healing Pipeline

node "Хост-машина (Windows/Linux)" {
    node "Docker Engine" {
        node "Airflow Container (standalone)" as AirflowContainer {
            component "Airflow Scheduler/Webserver" as AirflowSvc
            component "DAG-и: self_healing_pipeline,\nbusiness_analysis_pipeline" as Dags
            folder "/opt/airflow (mount)" as AirflowMount
        }
    }
    node "Ollama Runtime (host process)" as OllamaHost {
        component "Ollama Server (REST API :11434)" as OllamaSvc
        database "llama3.2 (GGUF)" as ModelFile
    }
    node ".NET Runtime (dotnet run)" as DotnetHost {
        component "ASP.NET Core Backend (Dashboard)" as Backend
        database "auth.db (SQLite)" as AuthDb
        folder "output/ (JSON health-звіти)" as OutputFolder
    }
    database "DuckDB файл (yelp.duckdb)" as DuckDbFile
}

node "Браузер користувача" as Browser {
    component "Blazor WebAssembly Client" as BlazorWasm
}

AirflowContainer --> DuckDbFile : DuckDB.NET (файловий доступ)
AirflowContainer --> OutputFolder : запис JSON health-звітів
AirflowContainer ..> OllamaSvc : REST API, HTTP (host.docker.internal:11434)
OllamaSvc --> ModelFile : inference (llama.cpp)

Backend --> DuckDbFile : SQL (пошук бізнесів)
Backend --> AuthDb : ADO.NET (облікові записи)
Backend --> OutputFolder : читання JSON-результатів
Backend ..> AirflowContainer : Airflow REST API v2 (тригер DAG, статус, логи)

Browser --> Backend : HTTPS (REST API, статичні файли WASM)
BlazorWasm --> Browser : виконується в браузері
@enduml

```

## Додаток 5. Sequence Diagram

```
@startuml SequenceDiagram_BusinessAnalysis
title Sequence Diagram - запуск business_analysis_pipeline через Blazor-дашборд

actor "Data/Business Analyst" as User
participant "Blazor Client (BusinessAnalysis.razor)" as Client
participant "Dashboard Backend (BusinessAnalysisController)" as Backend
database "DuckDB" as DuckDB
participant "Dashboard Backend (AirflowController)" as AirflowCtrl
participant "Airflow Orchestrator (business_analysis_pipeline)" as Airflow
participant "Self-Healing Module" as Healer
participant "Sentiment Module" as Sentiment
participant "Ollama (llama3.2)" as Ollama
database "output/ (JSON)" as Output

User -> Client : Пошук бізнесу (debounce 400мс)
Client -> Backend : GET /api/business-analysis/businesses?query=...
Backend -> DuckDB : SQL SELECT (пошук бізнесу)
DuckDB --> Backend : Список бізнесів
Backend --> Client : Результати пошуку

User -> Client : Натискає "Витягнути відгуки"
Client -> Backend : POST /api/business-analysis/export/{id}
Backend -> DuckDB : SQL SELECT (відгуки бізнесу)
DuckDB --> Backend : Набір відгуків
Backend -> Backend : Генерація NDJSON-файлу
Backend --> Client : Файл готовий (input/business-analysis_{id}.json)

User -> Client : Налаштовує параметри, натискає "Запустити"
Client -> AirflowCtrl : POST /api/airflow/trigger-dag
AirflowCtrl -> Airflow : Trigger DAG run (Param API)
Airflow --> AirflowCtrl : run_id

loop кожна задача DAG (6 кроків)
    Airflow -> Healer : diagnose_and_heal_batch
    Healer --> Airflow : healed-записи
    Airflow -> Sentiment : batch_analyze_sentiment
    Sentiment -> Ollama : POST /api/chat (промпт)
    Ollama --> Sentiment : {sentiment, confidence}
    Sentiment --> Airflow : класифіковані записи
end

Airflow -> Output : запис health-звіту (JSON)

loop PeriodicTimer (5 сек, polling)
    Client -> AirflowCtrl : GET /api/airflow/dag-run/{runId}
    AirflowCtrl -> Airflow : запит статусу
    Airflow --> AirflowCtrl : статус, прогрес
    AirflowCtrl --> Client : оновлення прогрес-бара
end

Client -> Backend : GET /api/analytics/summary/{businessId}
Backend -> Output : читання JSON-результату
Output --> Backend : health-звіт
Backend --> Client : Аналітика (графіки, таблиці)
Client --> User : Відображення результатів
@enduml
```